

Exercise 1: Inventory Management System

Understand the Problem

Why data structures and algorithms matter for large inventories: A warehouse can hold thousands or millions of distinct products, and the system must support frequent additions, updates, deletions, and lookups in real time. Without an appropriate data structure, operations like finding a product or checking stock levels would require scanning every record, which becomes far too slow as the inventory grows. Well-chosen data structures and algorithms keep these operations fast and predictable, control memory use, and make the system scalable as the business grows.

Suitable data structures for this problem:

- ArrayList – a dynamic array that is simple to use and good when products are mostly accessed by position or iterated over in order, but searching, updating, or deleting a specific product by ID is $O(n)$ because it requires a linear scan.
- HashMap – stores products keyed by productId, giving average $O(1)$ time for add, update, and delete by key, which is ideal when products are primarily looked up by their unique ID.
- Balanced Binary Search Tree / TreeMap – useful if products need to stay sorted (e.g., by productId or price) while still supporting $O(\log n)$ operations.

For this exercise, a `HashMap<String, Product>` keyed by productId is chosen, since the dominant operations are add, update, and delete by a unique product identifier.

Setup

A simple Java project (e.g., a Maven or plain Java project named `InventoryManagementSystem`) is created with a single package containing the `Product` class and an `InventoryManager` class.

Implementation

Product.java

```
public class Product {
    private String productId;
    private String productName;
    private int quantity;
    private double price;

    public Product(String productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    public String getProductId() { return productId; }
```

```

    public String getProductName() { return productName; }
    public int getQuantity()      { return quantity; }
    public double getPrice()      { return price; }

    public void setProductName(String productName) { this.productName = productName; }
    public void setQuantity(int quantity)         { this.quantity = quantity; }
    public void setPrice(double price)            { this.price = price; }

    @Override
    public String toString() {
        return String.format("%s | %s | qty=%d | $%.2f",
                               productId, productName, quantity, price);
    }
}

```

InventoryManager.java

```

import java.util.HashMap;
import java.util.Collection;

public class InventoryManager {
    private HashMap<String, Product> products = new HashMap<>();

    // Add a product - O(1) average
    public void addProduct(Product product) {
        products.put(product.getProductId(), product);
    }

    // Update a product's quantity and price - O(1) average
    public boolean updateProduct(String productId, int newQuantity, double newPrice) {
        Product existing = products.get(productId);
        if (existing == null) return false;
        existing.setQuantity(newQuantity);
        existing.setPrice(newPrice);
        return true;
    }

    // Delete a product - O(1) average
    public boolean deleteProduct(String productId) {
        return products.remove(productId) != null;
    }

    public Product getProduct(String productId) {
        return products.get(productId);
    }

    public Collection<Product> getAllProducts() {
        return products.values();
    }

    public static void main(String[] args) {
        InventoryManager manager = new InventoryManager();
        manager.addProduct(new Product("P001", "Wireless Mouse", 150, 12.99));
        manager.addProduct(new Product("P002", "USB-C Cable", 300, 5.49));
    }
}

```

```

manager.updateProduct("P001", 140, 11.99);
System.out.println(manager.getProduct("P001"));

manager.deleteProduct("P002");
System.out.println("Total products: " + manager.getAllProducts().size());
}
}

```

Analysis

Time complexity of each operation using a HashMap:

- Add: $O(1)$ average, $O(n)$ worst case if many keys hash to the same bucket and the map must resize/rehash.
- Update: $O(1)$ average – a get followed by field mutation, both hash-based lookups.
- Delete: $O(1)$ average – removing an entry by key is a direct hash-bucket operation.

By comparison, an ArrayList of products would require $O(n)$ for update and delete (since the matching product must first be found by scanning), making HashMap far more efficient when productId-based access dominates.

Optimization strategies:

- Choose an initial capacity and load factor for the HashMap that minimizes resizing/rehashing for the expected inventory size.
- Use a well-distributed hashCode() (Strings already hash well) to avoid clustering and bucket collisions.
- If range queries or sorted iteration are also needed frequently (e.g., low-stock reports), maintain a secondary index such as a TreeMap on quantity, accepting the extra memory for faster reads.
- Batch updates where possible to reduce repeated lookups, and consider caching frequently accessed products.

Output

```

4 public class InventoryManager {
22     public boolean deleteProduct(String productId) {
23         return products.remove(productId) != null;
24     }
25
26     public Product getProduct(String productId) {
27         return products.get(productId);
28     }
29
30     public Collection<Product> getAllProducts() {

```

```

P001 | Wireless Mouse | qty=140 | $11.99
Total products: 1

[Done] exited with code=0 in 1.031 seconds

[Running] cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data
Structures\Exercise1_InventoryManagement\" && javac InventoryManager.java && java InventoryManager
P001 | Wireless Mouse | qty=140 | $11.99
Total products: 1

[Done] exited with code=0 in 0.803 seconds

[Running] cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data
Structures\Exercise1_InventoryManagement\" && javac InventoryManager.java && java InventoryManager
P001 | Wireless Mouse | qty=140 | $11.99
Total products: 1

[Done] exited with code=0 in 0.826 seconds

```

Exercise 2: E-commerce Platform Search Function

Understand Asymptotic Notation

Big O notation describes how the running time (or memory use) of an algorithm grows as the input size n grows, focusing on the dominant term and ignoring constant factors and lower-order terms. It gives an upper bound on growth rate, allowing algorithms to be compared independently of hardware or implementation details, which is essential for predicting how a search function will behave as the product catalog scales from hundreds to millions of items.

Best, average, and worst case for search operations:

- Best case: the target is found almost immediately (e.g., the first element checked, or the middle element in binary search) – $O(1)$.
- Average case: the target is found after inspecting roughly half the data structure on average (linear search) or after a typical number of halving steps (binary search).
- Worst case: the target is not present, or is positioned such that the maximum number of comparisons is required – $O(n)$ for linear search, $O(\log n)$ for binary search.

Setup

Product.java

```
public class Product {
    private String productId;
    private String productName;
    private String category;

    public Product(String productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    public String getProductId() { return productId; }
    public String getProductName() { return productName; }
    public String getCategory() { return category; }

    @Override
    public String toString() {
        return productId + " | " + productName + " | " + category;
    }
}
```

Implementation

SearchService.java

```
public class SearchService {

    // Linear search - works on an unsorted (or sorted) array
    public static Product linearSearch(Product[] products, String productName) {
        for (Product product : products) {
            if (product.getProductName().equalsIgnoreCase(productName)) {
                return product;
            }
        }
        return null;
    }

    // Binary search - requires the array to be sorted by productName
    public static Product binarySearch(Product[] sortedProducts, String productName) {
        int low = 0, high = sortedProducts.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            int cmp =
sortedProducts[mid].getProductName().compareToIgnoreCase(productName);
            if (cmp == 0) {
                return sortedProducts[mid];
            } else if (cmp < 0) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return null;
    }

    public static void main(String[] args) {
        Product[] products = {
            new Product("P003", "Bluetooth Speaker", "Electronics"),
            new Product("P001", "Desk Lamp", "Home"),
            new Product("P002", "Notebook", "Stationery"),
        };

        System.out.println("Linear: " + linearSearch(products, "Notebook"));

        Product[] sorted = products.clone();
        java.util.Arrays.sort(sorted, (a, b) ->
a.getProductName().compareToIgnoreCase(b.getProductName()));
        System.out.println("Binary: " + binarySearch(sorted, "Notebook"));
    }
}
```

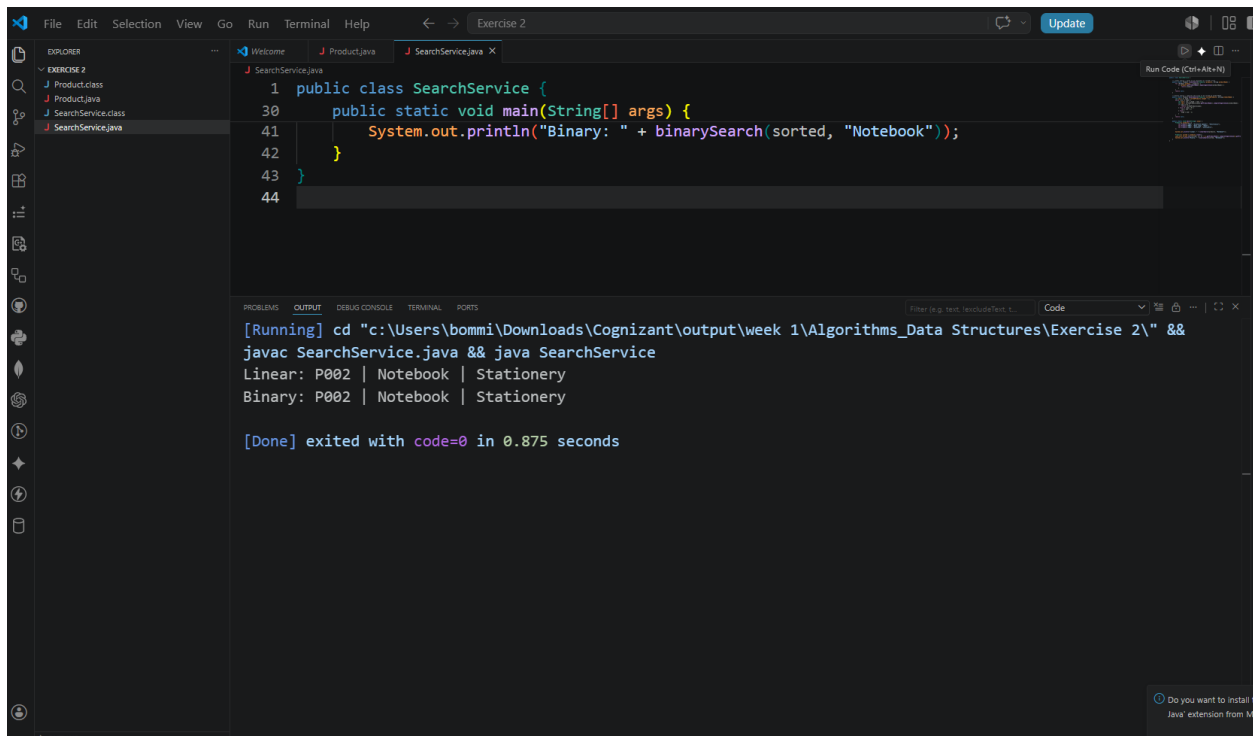
Analysis

- Linear search: $O(1)$ best case, $O(n)$ average and worst case – every element may need to be checked, and no ordering assumption is required.

- Binary search: $O(1)$ best case, $O(\log n)$ average and worst case – each comparison eliminates half of the remaining search space, but the array must be sorted first (sorting itself costs $O(n \log n)$).

Binary search is far more suitable for the e-commerce platform's search function once the catalog is sorted (e.g., maintained in sorted order or indexed), since $O(\log n)$ scales dramatically better than $O(n)$ for large catalogs – searching a million products takes roughly 20 comparisons instead of up to a million. In practice, a real search engine would also use indexing structures such as a hash index or a search index (e.g., a trie or inverted index for text search), but among the two algorithms studied, binary search is the clear choice whenever the data can be kept sorted.

Output



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'EXERCISE 2' with files 'Product.class', 'Product.java', 'SearchService.class', and 'SearchService.java'. The code editor displays the following Java code:

```
1 public class SearchService {  
30     public static void main(String[] args) {  
41         System.out.println("Binary: " + binarySearch(sorted, "Notebook"));  
42     }  
43 }  
44
```

The terminal at the bottom shows the execution of the program:

```
[Running] cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data Structures\Exercise 2\" &&  
javac SearchService.java && java SearchService  
Linear: P002 | Notebook | Stationery  
Binary: P002 | Notebook | Stationery  
  
[Done] exited with code=0 in 0.875 seconds
```

A small notification in the bottom right corner asks: 'Do you want to install the Java extension from Microsoft?'.

Exercise 3: Sorting Customer Orders

Understand Sorting Algorithms

- Bubble Sort: repeatedly steps through the list, comparing adjacent elements and swapping them if they are in the wrong order. After each full pass, the largest unsorted element “bubbles” to its correct position. Simple but inefficient – $O(n^2)$ in the average and worst case.
- Insertion Sort: builds the sorted portion of the array one element at a time, inserting each new element into its correct position relative to the already-sorted portion. Efficient for small or nearly-sorted data – $O(n^2)$ worst case, $O(n)$ best case.
- Quick Sort: a divide-and-conquer algorithm that selects a pivot, partitions the array so smaller elements are to the left and larger to the right, then recursively sorts each partition. $O(n \log n)$ on average, $O(n^2)$ worst case (rare with a good pivot strategy).
- Merge Sort: another divide-and-conquer algorithm that splits the array in half, recursively sorts each half, then merges the two sorted halves. Guarantees $O(n \log n)$ in all cases, at the cost of $O(n)$ extra space.

Setup

Order.java

```
public class Order {
    private String orderId;
    private String customerName;
    private double totalPrice;

    public Order(String orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }

    public String getOrderId()      { return orderId; }
    public String getCustomerName() { return customerName; }
    public double getTotalPrice()   { return totalPrice; }

    @Override
    public String toString() {
        return orderId + " | " + customerName + " | $" + totalPrice;
    }
}
```

Implementation

OrderSorter.java

```
public class OrderSorter {

    // Bubble Sort by totalPrice (ascending)
    public static void bubbleSort(Order[] orders) {
        int n = orders.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - 1 - i; j++) {
                if (orders[j].getTotalPrice() > orders[j + 1].getTotalPrice()) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break; // already sorted, stop early
        }
    }

    // Quick Sort by totalPrice (ascending)
    public static void quickSort(Order[] orders, int low, int high) {
        if (low < high) {
            int pivotIndex = partition(orders, low, high);
            quickSort(orders, low, pivotIndex - 1);
            quickSort(orders, pivotIndex + 1, high);
        }
    }

    private static int partition(Order[] orders, int low, int high) {
        double pivot = orders[high].getTotalPrice();
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (orders[j].getTotalPrice() <= pivot) {
                i++;
                Order temp = orders[i];
                orders[i] = orders[j];
                orders[j] = temp;
            }
        }
        Order temp = orders[i + 1];
        orders[i + 1] = orders[high];
        orders[high] = temp;
        return i + 1;
    }

    public static void main(String[] args) {
        Order[] orders1 = {
            new Order("O1", "Alice", 250.0),
            new Order("O2", "Bob", 80.0),
            new Order("O3", "Carlos", 430.0),
            new Order("O4", "Dina", 120.0),
        };
    }
}
```



```

        Order[] orders2 = orders1.clone();

        bubbleSort(orders1);
        System.out.println("Bubble sorted: " + java.util.Arrays.toString(orders1));

        quickSort(orders2, 0, orders2.length - 1);
        System.out.println("Quick sorted: " + java.util.Arrays.toString(orders2));
    }
}

```

Analysis

- Bubble Sort: $O(n)$ best case (already sorted, with the early-exit flag), $O(n^2)$ average and worst case. It performs many redundant comparisons and swaps, making it impractical for large order lists.
- Quick Sort: $O(n \log n)$ average case, $O(n^2)$ worst case (e.g., already-sorted input with a poor pivot choice), but in practice consistently outperforms Bubble Sort because of its divide-and-conquer structure and good cache locality.

Quick Sort is generally preferred over Bubble Sort because its average-case $O(n \log n)$ complexity scales much better as the number of orders grows – sorting 10,000 orders with Bubble Sort requires on the order of 100 million comparisons, whereas Quick Sort needs roughly 130,000. Quick Sort also sorts in place with low memory overhead, unlike Merge Sort. The main caveat is Quick Sort's $O(n^2)$ worst case, which can be mitigated by randomizing the pivot or choosing the median of three candidate values.

Output

```

1 public class OrderSorter {
46     public static void main(String[] args) {
59         System.out.println("Quick sorted: " + java.util.Arrays.toString(orders2));
60     }
61 }
62

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

[Running] cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data Structures\Exercise 3\" &&
javac OrderSorter.java && java OrderSorter
Bubble sorted: [02 | Bob | $80.0, 04 | Dina | $120.0, 01 | Alice | $250.0, 03 | Carlos | $430.0]
Quick sorted: [02 | Bob | $80.0, 04 | Dina | $120.0, 01 | Alice | $250.0, 03 | Carlos | $430.0]

[Done] exited with code=0 in 0.844 seconds

```

Do you want to install 'Java' extension from M...

Exercise 4: Employee Management System

Understand Array Representation

An array is a fixed-size, contiguous block of memory in which each element is stored back-to-back and accessed using an index. Because the memory is contiguous and every element has the same size, the address of any element can be computed directly from the base address and the index ($\text{base} + \text{index} \times \text{elementSize}$), giving $O(1)$ random access. Advantages of arrays include constant-time indexed access, excellent cache locality (elements are stored next to each other, so iterating over them is fast), and low memory overhead compared to linked structures, since no extra pointers are stored per element.

Setup

Employee.java

```
public class Employee {
    private String employeeId;
    private String name;
    private String position;
    private double salary;

    public Employee(String employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }

    public String getEmployeeId() { return employeeId; }
    public String getName()       { return name; }
    public String getPosition()   { return position; }
    public double getSalary()     { return salary; }

    @Override
    public String toString() {
        return employeeId + " | " + name + " | " + position + " | $" + salary;
    }
}
```

Implementation

EmployeeArrayManager.java

```
import java.util.Arrays;

public class EmployeeArrayManager {
    private Employee[] employees;
    private int size; // number of employees currently stored
}
```

```

public EmployeeArrayManager(int capacity) {
    employees = new Employee[capacity];
    size = 0;
}

// Add - O(1) amortized O(n) if the array must grow
public boolean add(Employee employee) {
    if (size == employees.length) {
        employees = Arrays.copyOf(employees, employees.length * 2); // grow - O(n)
    }
    employees[size++] = employee;
    return true;
}

// Search by employeeId - O(n)
public Employee search(String employeeId) {
    for (int i = 0; i < size; i++) {
        if (employees[i].getEmployeeId().equals(employeeId)) {
            return employees[i];
        }
    }
    return null;
}

// Traverse - O(n)
public void traverse() {
    for (int i = 0; i < size; i++) {
        System.out.println(employees[i]);
    }
}

// Delete by employeeId - O(n)
public boolean delete(String employeeId) {
    for (int i = 0; i < size; i++) {
        if (employees[i].getEmployeeId().equals(employeeId)) {
            // shift subsequent elements left to fill the gap
            for (int j = i; j < size - 1; j++) {
                employees[j] = employees[j + 1];
            }
            employees[--size] = null;
            return true;
        }
    }
    return false;
}

public static void main(String[] args) {
    EmployeeArrayManager manager = new EmployeeArrayManager(4);
    manager.add(new Employee("E1", "Maya", "Engineer", 75000));
    manager.add(new Employee("E2", "Liam", "Designer", 68000));

    manager.traverse();
    System.out.println("Found: " + manager.search("E2"));
    manager.delete("E1");
    manager.traverse();
}

```

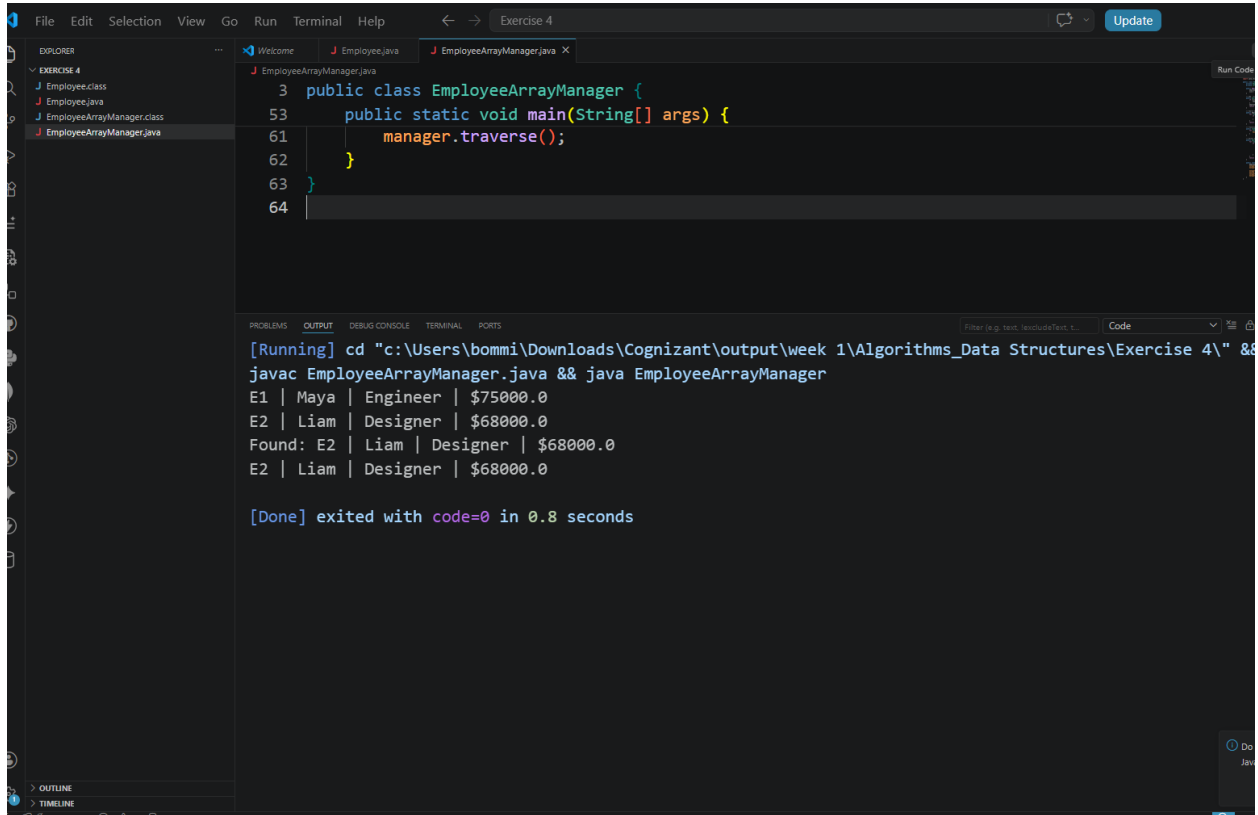
```
}  
}
```

Analysis

- Add: $O(1)$ amortized when there is spare capacity; $O(n)$ when the underlying array must be resized and all elements copied.
- Search: $O(n)$ – in the worst case every element must be checked, since the array is not sorted or indexed by employeeid.
- Traverse: $O(n)$ – visiting every element exactly once.
- Delete: $O(n)$ – finding the element is $O(n)$, and removing it requires shifting all subsequent elements left by one position to keep the array contiguous.

Limitations of arrays: fixed initial size, requiring costly resizing (copying all elements) as the system grows; insertion and deletion in the middle are expensive due to the need to shift elements; and searching by a field other than index requires a linear scan unless the array is sorted or paired with an auxiliary index. Arrays are best used when the dataset size is known in advance or changes infrequently, when fast indexed/positional access is the priority, and when memory efficiency and cache performance matter more than frequent insertions and deletions. For an employee system with frequent additions and removals, a structure such as a HashMap (for fast lookup by ID) or a linked list (for cheap insert/delete) is often a better fit.

Output



The screenshot shows an IDE with a Java file named `EmployeeArrayManager.java`. The code defines a class `EmployeeArrayManager` with a `main` method that calls `manager.traverse()`. The output window shows the execution of the program, which prints employee data and finds a specific employee.

```
public class EmployeeArrayManager {  
    public static void main(String[] args) {  
        manager.traverse();  
    }  
}
```

[Running] `cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data Structures\Exercise 4\" && javac EmployeeArrayManager.java && java EmployeeArrayManager`

```
E1 | Maya | Engineer | $75000.0  
E2 | Liam | Designer | $68000.0  
Found: E2 | Liam | Designer | $68000.0  
E2 | Liam | Designer | $68000.0
```

[Done] exited with code=0 in 0.8 seconds

Exercise 5: Task Management System

Understand Linked Lists

- **Singly Linked List:** each node stores data plus a single reference (“next”) pointing to the following node. Traversal is only possible in the forward direction. It uses less memory per node than a doubly linked list.
- **Doubly Linked List:** each node stores data plus two references, “next” and “prev”, allowing traversal in both directions and making deletion of a known node $O(1)$ without needing to find its predecessor.

Setup

Task.java

```
public class Task {
    private String taskId;
    private String taskName;
    private String status;

    public Task(String taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
    }

    public String getTaskId()    { return taskId; }
    public String getTaskName() { return taskName; }
    public String getStatus()    { return status; }

    @Override
    public String toString() {
        return taskId + " | " + taskName + " | " + status;
    }
}
```

Implementation

TaskLinkedList.java

```
public class TaskLinkedList {

    private static class Node {
        Task task;
        Node next;
        Node(Task task) { this.task = task; }
    }

    private Node head;
    private int size;
```

```

// Add to the end of the list - O(n) (O(1) if a tail pointer is maintained)
public void add(Task task) {
    Node newNode = new Node(task);
    if (head == null) {
        head = newNode;
    } else {
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    size++;
}

// Search by taskId - O(n)
public Task search(String taskId) {
    Node current = head;
    while (current != null) {
        if (current.task.getTaskId().equals(taskId)) {
            return current.task;
        }
        current = current.next;
    }
    return null;
}

// Traverse - O(n)
public void traverse() {
    Node current = head;
    while (current != null) {
        System.out.println(current.task);
        current = current.next;
    }
}

// Delete by taskId - O(n)
public boolean delete(String taskId) {
    if (head == null) return false;
    if (head.task.getTaskId().equals(taskId)) {
        head = head.next;
        size--;
        return true;
    }
    Node current = head;
    while (current.next != null) {
        if (current.next.task.getTaskId().equals(taskId)) {
            current.next = current.next.next;
            size--;
            return true;
        }
        current = current.next;
    }
    return false;
}

```

```

public static void main(String[] args) {
    TaskLinkedList tasks = new TaskLinkedList();
    tasks.add(new Task("T1", "Design schema", "In Progress"));
    tasks.add(new Task("T2", "Write tests", "Pending"));

    tasks.traverse();
    System.out.println("Found: " + tasks.search("T2"));
    tasks.delete("T1");
    tasks.traverse();
}
}

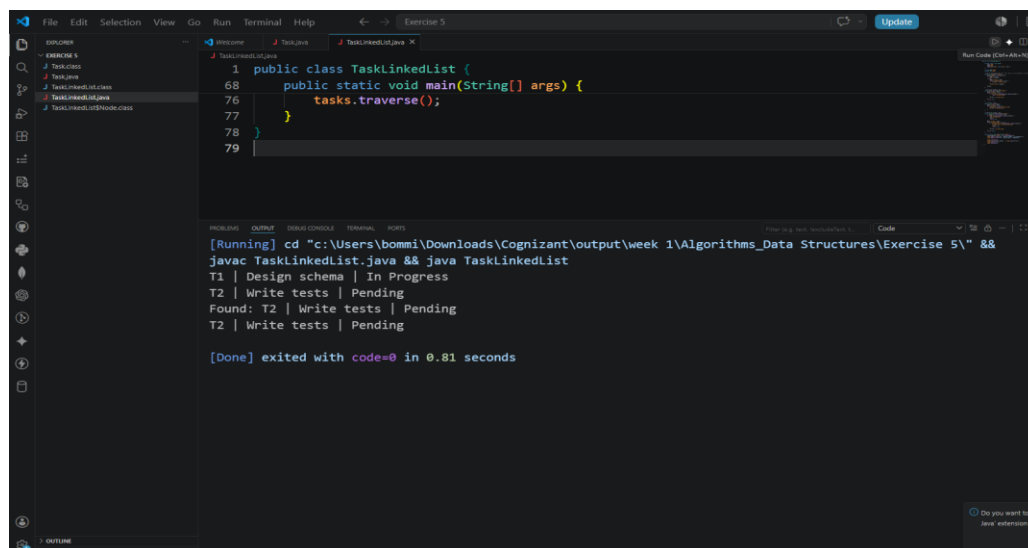
```

Analysis

- Add: $O(n)$ to reach the end of the list without a tail pointer ($O(1)$ if a tail reference is maintained).
- Search: $O(n)$ – nodes must be visited sequentially from the head since there is no random access.
- Traverse: $O(n)$ – visiting each node exactly once.
- Delete: $O(n)$ to locate the node (plus its predecessor), but $O(1)$ to actually unlink it once found, since no shifting of other elements is required.

Advantages of linked lists over arrays for dynamic data: a linked list grows and shrinks one node at a time without ever needing to allocate a new contiguous block and copy existing elements, unlike a resizing array. Inserting or deleting at a known position (e.g., the head, or a node already being traversed) is $O(1)$ for a linked list because no shifting of subsequent elements is needed, whereas an array requires $O(n)$ shifting. The trade-off is that linked lists use extra memory for the next/prev references and lose the $O(1)$ random-access and cache-locality advantages that arrays provide, since elements are scattered in memory and indexed access requires $O(n)$ traversal.

Output



The screenshot shows an IDE with a file explorer on the left containing files like `EXERCISE 5`, `TaskLinkList`, `TaskLinkListNode`, and `TaskLinkListNode.class`. The main editor displays the `TaskLinkedList.java` code, which includes a `public class TaskLinkedList` with a `public static void main` method. The terminal at the bottom shows the command `cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data Structures\Exercise 5\" && java TaskLinkList` and its output: `T1 | Design schema | In Progress`, `T2 | Write tests | Pending`, `Found: T2 | Write tests | Pending`, and `T2 | Write tests | Pending`. The terminal also indicates `[Done] exited with code=0 in 0.81 seconds`.

Exercise 6: Library Management System

Understand Search Algorithms

Linear search checks each element of a collection in turn, comparing it to the target, until a match is found or the collection is exhausted. It works on any collection regardless of ordering, but in the worst case requires inspecting every element – $O(n)$.

Binary search repeatedly halves the search interval: it compares the target with the middle element of a sorted collection and discards the half that cannot contain the target, repeating until the element is found or the interval is empty. Because each comparison eliminates half of the remaining candidates, its time complexity is $O(\log n)$, but it strictly requires the underlying data to be sorted.

Setup

Book.java

```
public class Book {
    private String bookId;
    private String title;
    private String author;

    public Book(String bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    public String getBookId() { return bookId; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }

    @Override
    public String toString() {
        return bookId + " | " + title + " | " + author;
    }
}
```

Implementation

LibrarySearch.java

```
import java.util.Arrays;
import java.util.Comparator;

public class LibrarySearch {

    // Linear search by title - works on any (sorted or unsorted) array
    public static Book linearSearchByTitle(Book[] books, String title) {
        for (Book book : books) {
```



```

        if (book.getTitle().equalsIgnoreCase(title)) {
            return book;
        }
    }
    return null;
}

// Binary search by title - requires the array to be sorted by title
public static Book binarySearchByTitle(Book[] sortedBooks, String title) {
    int low = 0, high = sortedBooks.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int cmp = sortedBooks[mid].getTitle().compareToIgnoreCase(title);
        if (cmp == 0) {
            return sortedBooks[mid];
        } else if (cmp < 0) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return null;
}

public static void main(String[] args) {
    Book[] books = {
        new Book("B3", "The Hobbit", "J.R.R. Tolkien"),
        new Book("B1", "Dune", "Frank Herbert"),
        new Book("B2", "Foundation", "Isaac Asimov"),
    };

    System.out.println("Linear: " + linearSearchByTitle(books, "Foundation"));

    Book[] sorted = books.clone();
    Arrays.sort(sorted, Comparator.comparing(Book::getTitle,
String.CASE_INSENSITIVE_ORDER));
    System.out.println("Binary: " + binarySearchByTitle(sorted, "Foundation"));
}
}

```

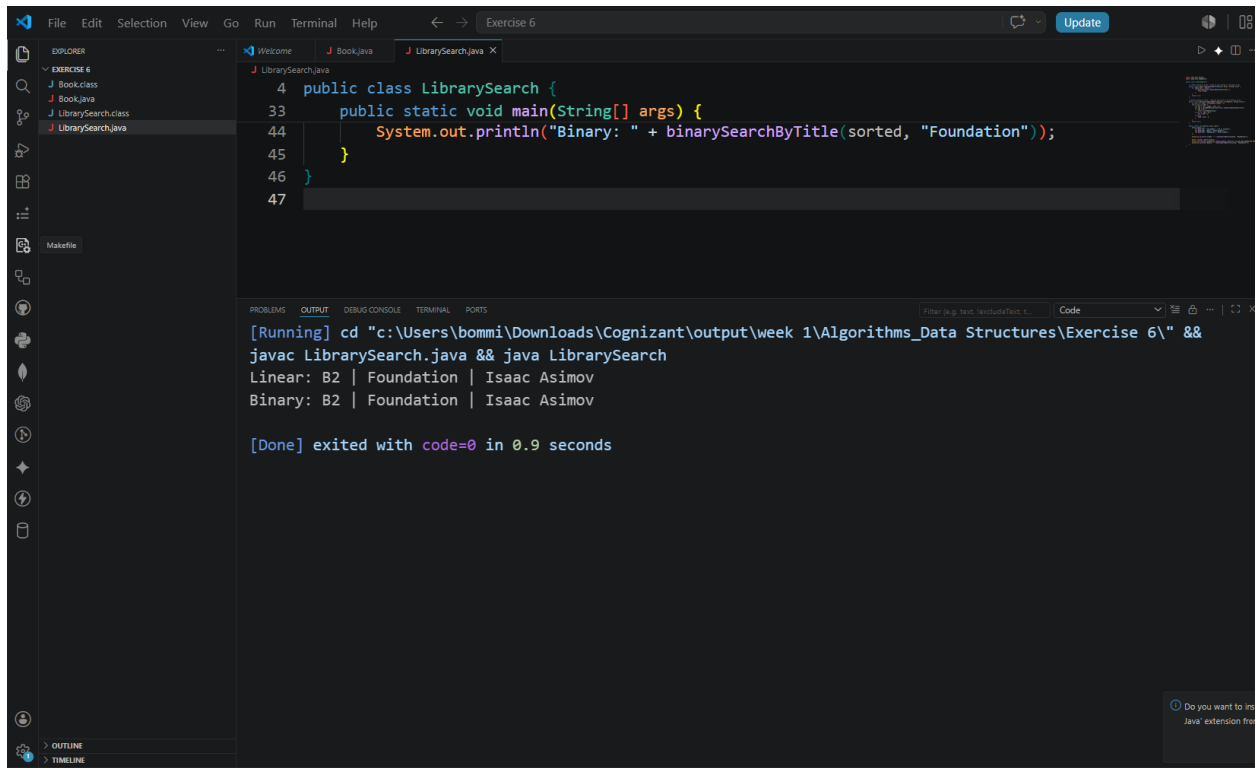
Analysis

- Linear search: $O(n)$ worst and average case, $O(1)$ best case; no sorting required, simple to implement, and well-suited to small or rarely-searched collections.
- Binary search: $O(\log n)$ worst and average case, $O(1)$ best case; requires the data to be sorted, and resorting after every new book is added or removed adds overhead ($O(n \log n)$ per sort, or $O(n)$ per insertion to keep it sorted).

When to use each algorithm: linear search is preferable for small catalogs, data that changes frequently and is expensive to keep sorted, or when the data structure does not support efficient random access (such as a linked list). Binary search is preferable for large, relatively static or infrequently updated catalogs (e.g., a library's main catalog refreshed periodically) where the dataset can be kept sorted by title, since its $O(\log n)$

performance scales dramatically better – searching among a million titles takes about 20 comparisons rather than up to a million.

Output



The screenshot shows an IDE with a dark theme. The Explorer panel on the left shows a project named 'EXERCISE 6' with files 'Book.class', 'Book.java', 'LibrarySearch.class', and 'LibrarySearch.java'. The main editor displays the 'LibrarySearch.java' file with the following code:

```
4 public class LibrarySearch {
33     public static void main(String[] args) {
44         System.out.println("Binary: " + binarySearchByTitle(sorted, "Foundation"));
45     }
46 }
47
```

The Output panel at the bottom shows the execution results:

```
[Running] cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data Structures\Exercise 6\" &&
javac LibrarySearch.java && java LibrarySearch
Linear: B2 | Foundation | Isaac Asimov
Binary: B2 | Foundation | Isaac Asimov

[Done] exited with code=0 in 0.9 seconds
```

A small notification in the bottom right corner asks: 'Do you want to ins Java' extension fro'.

Exercise 7: Financial Forecasting

Understand Recursive Algorithms

Recursion is a technique where a method solves a problem by calling itself on a smaller version of the same problem, combined with one or more base cases that stop the recursion. It simplifies problems that have a naturally repetitive or self-similar structure – such as computing compound growth year over year – by expressing the solution in terms of a smaller instance of itself, rather than manually tracking loop state. This often produces code that mirrors the mathematical definition of the problem and is easier to reason about, at the cost of additional function-call overhead and stack usage.

Setup

FinancialForecaster.java – method signature

```
// Calculates the future value of an amount after n periods,  
// growing by a fixed growth rate each period, using recursion.  
public static double futureValue(double presentValue, double growthRate, int periods)  
{  
    // recursive declaration shown in full implementation below  
}
```

Implementation

FinancialForecaster.java

```
public class FinancialForecaster {  
  
    // Recursive calculation of future value:  
    // futureValue(P, r, 0) = P  
    // futureValue(P, r, n) = futureValue(P, r, n - 1) * (1 + r)  
    public static double futureValue(double presentValue, double growthRate, int  
periods) {  
        if (periods == 0) { // base case  
            return presentValue;  
        }  
        return futureValue(presentValue, growthRate, periods - 1) * (1 + growthRate);  
    }  
  
    // Optimized version using memoization to avoid recomputation  
    // when forecasting multiple overlapping horizons.  
    public static double futureValueMemo(double presentValue, double growthRate, int  
periods,  
                                         java.util.Map<Integer, Double> cache) {  
        if (periods == 0) return presentValue;  
        if (cache.containsKey(periods)) return cache.get(periods);  
        double result = futureValueMemo(presentValue, growthRate, periods - 1, cache)  
* (1 + growthRate);  
        cache.put(periods, result);  
        return result;  
    }  
}
```

```

public static void main(String[] args) {
    double presentValue = 10000.0;
    double growthRate = 0.05; // 5% growth per period
    int periods = 10;

    double result = futureValue(presentValue, growthRate, periods);
    System.out.printf("Forecast value after %d periods: $%.2f%n", periods,
result);

    java.util.Map<Integer, Double> cache = new java.util.HashMap<>();
    double resultMemo = futureValueMemo(presentValue, growthRate, periods, cache);
    System.out.printf("Memoized forecast: $%.2f%n", resultMemo);
}
}

```

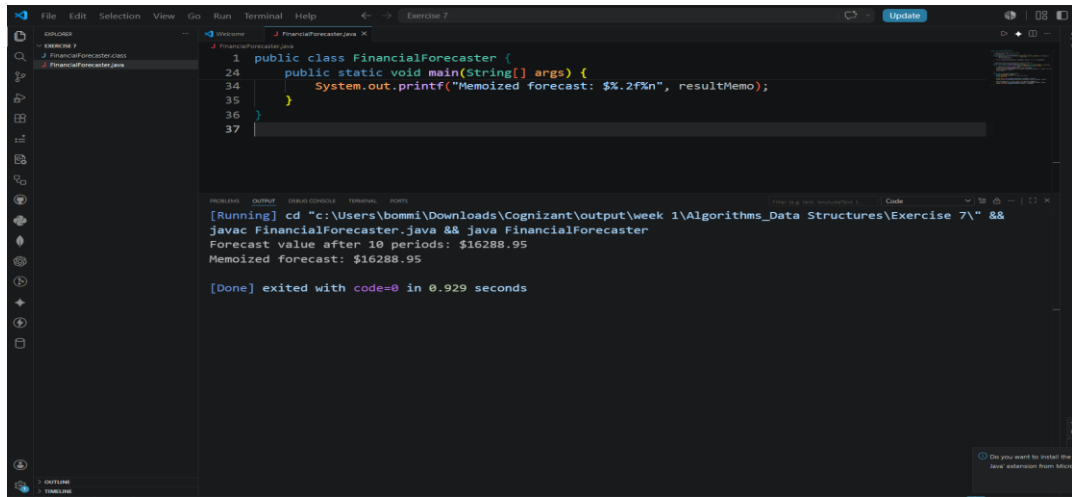
Analysis

Time complexity: the plain recursive `futureValue` method makes exactly one recursive call per period with $O(1)$ work per call, giving $O(n)$ time complexity and $O(n)$ space complexity due to the call stack (one stack frame per period, since this is not tail-call optimized by the JVM).

Optimizing the recursive solution:

- Memoization: cache previously computed results (as shown in `futureValueMemo`) so that repeated or overlapping forecast requests – e.g., asking for the value at period 10 after already having computed periods 1 through 8 – reuse prior work instead of recomputing it, which helps significantly when the same forecaster is queried for multiple horizons.
- Iteration: since this particular recursion is linear (one self-call per step) and not naturally tree-shaped, it can be rewritten as a simple loop or, more directly, with the closed-form formula $\text{presentValue} * (1 + \text{growthRate})^{\text{periods}}$, both of which run in $O(1)$ extra space and avoid recursion overhead entirely; this is generally the best optimization for this specific problem.
- Tail-call style restructuring can reduce conceptual stack depth, but Java does not perform tail-call optimization, so for very large numbers of periods an explicit loop remains the safest choice to avoid `StackOverflowError`.
-

Output



```
1 public class FinancialForecaster {
24     public static void main(String[] args) {
34         System.out.printf("Memoized forecast: %.2f\n", resultMemo);
35     }
36 }
37
```

[Running] cd "c:\Users\bommi\Downloads\Cognizant\output\week 1\Algorithms_Data Structures\Exercise 7\" &&
javac FinancialForecaster.java && java FinancialForecaster
Forecast value after 10 periods: \$16288.95
Memoized forecast: \$16288.95
[Done] exited with code=0 in 0.929 seconds